



# Seeing The Atom

An interactive visualization of hydrogen orbitals for students new to quantum chemistry

Student: Manav Sachdeva

Student Number: 49672835

Visualization, Computer  
Graphics and Data Analysis

## Contents

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 1. Introduction: the problem this tool solves .....             | 1  |
| 2. The tool: what a student sees .....                          | 2  |
| 2.1 First impressions.....                                      | 2  |
| 2.2 Choosing an orbital.....                                    | 2  |
| 2.3 Three ways to see the same orbital .....                    | 2  |
| 2.4 The two supporting plots .....                              | 3  |
| 2.5 A short guided tour.....                                    | 4  |
| 3. Visualisation design .....                                   | 6  |
| 3.1 Why three render modes .....                                | 6  |
| 3.2 Why encode phase in colour .....                            | 6  |
| 3.3 Why three coordinated views .....                           | 7  |
| 3.4 Designing for the beginner .....                            | 7  |
| 4. Computer-graphics techniques used.....                       | 8  |
| 4.1 Turning a field into a surface: marching cubes .....        | 8  |
| 4.2 The glowing cloud: volume ray-marching .....                | 9  |
| 4.3 Making it look three-dimensional: lighting and shading..... | 9  |
| 4.4 Colour, transparency, and the point cloud.....              | 10 |
| 4.5 Coordinates, camera, and axis labels .....                  | 11 |
| 4.6 The 2D plots.....                                           | 12 |
| 5. Discussion .....                                             | 13 |
| 5.1 Comparing the three rendering methods .....                 | 13 |
| 5.2 Performance .....                                           | 13 |
| 5.3 Limitations and what was left out.....                      | 14 |
| 5.4 What I learned building it.....                             | 14 |
| 6. Self-assessment .....                                        | 15 |
| 6.1 Meeting the minimum requirements.....                       | 15 |
| 6.2 Evidence of advanced skills and understanding.....          | 15 |
| 6.3 What I would do differently .....                           | 16 |
| 7. Conclusion .....                                             | 17 |
| References.....                                                 | 17 |
| Appendix A: Code and How to Run it .....                        | 17 |
| Appendix B: List of figures.....                                | 18 |

## 1. Introduction: the problem this tool solves

Almost every chemistry and physics student meets atomic orbitals through the same small set of pictures: a round 1s, a dumbbell 2p, a four-lobed 3d. These cartoons are printed flat, viewed from one fixed angle, and rarely explain what the shapes mean. Students are left with two persistent misconceptions: that electrons travel along orbits the way planets circle the sun, and that an orbital is a solid object with a hard edge.

The reality is subtler and more interesting. An electron has no path. What we draw as an orbital is a probability cloud: the wavefunction describes how likely the electron is to be found at each point in space, and the familiar lobed shapes are just the regions where that probability is high. The shapes also carry a hidden property the sign, or phase, of the wavefunction which is invisible in a textbook picture but is what makes chemical bonding work.

This project is a visualisation built to close that gap for students who are just starting out. It is an interactive, browser-based tool that lets a learner choose any hydrogen orbital up to  $n = 3$ , rotate it freely, and view it three complementary ways: as a glowing probability cloud, as a clean solid surface, or as a scatter of points. Alongside the 3D view, two coordinated 2D plots show how the probability changes with distance from the nucleus and what the orbital looks like sliced open along a plane. The aim is for a student to come away with a genuine three-dimensional, intuition-level understanding of what an orbital is something a flat diagram cannot provide.

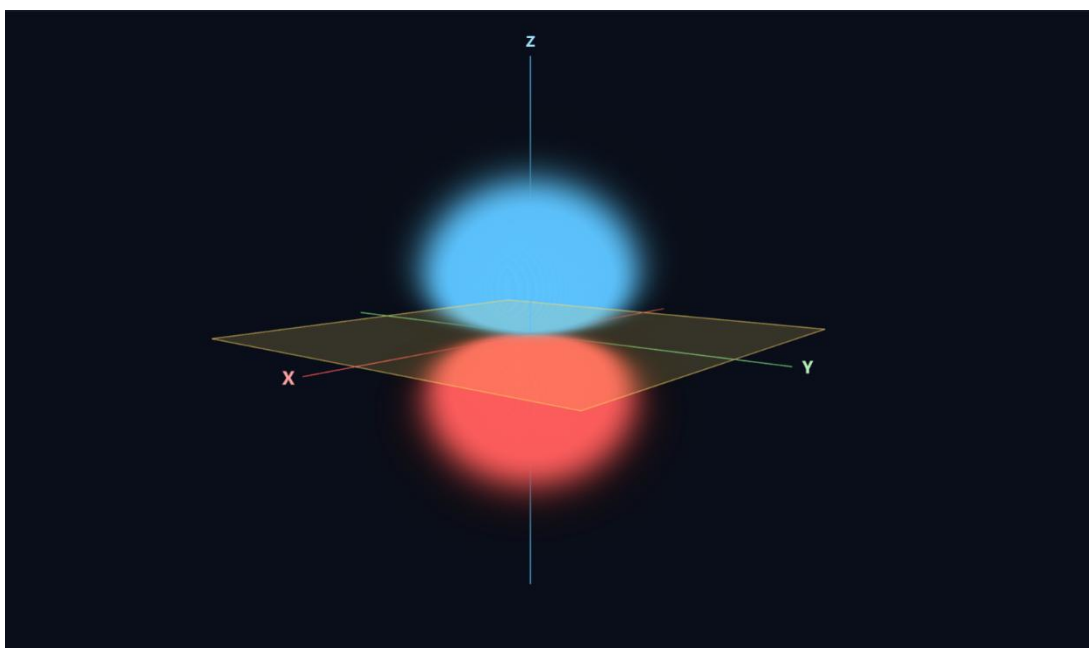
Although the goal is teaching, the tool is built entirely from computer-graphics techniques drawn from the course: surface extraction, volume raymarching, lighting and shading, colour encoding, transparency, and texturing. The visualisation is the purpose; the graphics are how it is achieved. This report describes both what the tool shows a learner, and the graphics methods that make it work.

## 2. The tool: what a student sees

This section is the communication document it presents the imagery and walks through the tool as a beginning student would experience it.

### 2.1 First impressions

On opening the tool, a student sees a single orbital floating in space, slowly invitingly rotatable with the mouse. The default is the 2p orbital aligned with the vertical axis: a blue lobe above and a red lobe below. The very first thing the colours teach is that the two lobes are not the same they have opposite phase. This is usually the first surprise for a learner, and it is visible immediately, before any explanation.



*Fig.1 The 2p orbital in volume mode (the two glowing lobes, blue above, red below), with the labelled axes visible.*

### 2.2 Choosing an orbital

A dropdown lets the student step through the orbitals in the order they would meet them in class: 1s, 2s, 2p, 3s, 3p, 3d. As they move up, the orbitals grow, gain lobes, and develop internal structure. The tool keeps the view framed and the axes labelled so the student is never lost.

Stepping through the dropdown gives a clear sense of how orbitals are organised. As the principal quantum number  $n$  increases ( $1s \rightarrow 2s \rightarrow 3s$ , or  $2p \rightarrow 3p$ ), the orbitals grow larger and develop additional structure inside, like a ripple expanding outward. As the angular momentum number  $l$  increases at fixed  $n$  (e.g.  $3s \rightarrow 3p \rightarrow 3d$ ), the orbital gains angular lobes the round  $s$  becomes the two-lobed  $p$ , then the four-lobed  $d$ . And as  $m$  varies at fixed  $n$  and  $l$ , the same shape simply reorients in space  $p_x$ ,  $p_y$ , and  $p_z$  are the same orbital pointing along three different axes. These three patterns size with  $n$ , lobes with  $l$ , orientation with  $m$  are the entire grammar of the orbital naming scheme, and the tool makes them visible at a glance.

### 2.3 Three ways to see the same orbital

The same orbital can be viewed three ways, and a large part of the tool's teaching value is in letting students switch between them and see that they are looking at one underlying thing.

- Volume: a soft glowing cloud, brightest where the electron is most likely to be found, fading gently to nothing. This is the most honest picture of a probability cloud and the best antidote to the hard-edged-object misconception.
- Isosurface: a clean solid shell drawn at a chosen probability level. This is the closest to the textbook picture, and a slider lets the student inflate or shrink the shell to see that the boundary is a choice, not a real edge.
- Point cloud: thousands of dots, denser where the electron is more likely to be. This makes the word probability concrete each dot is a possible place the electron could be.

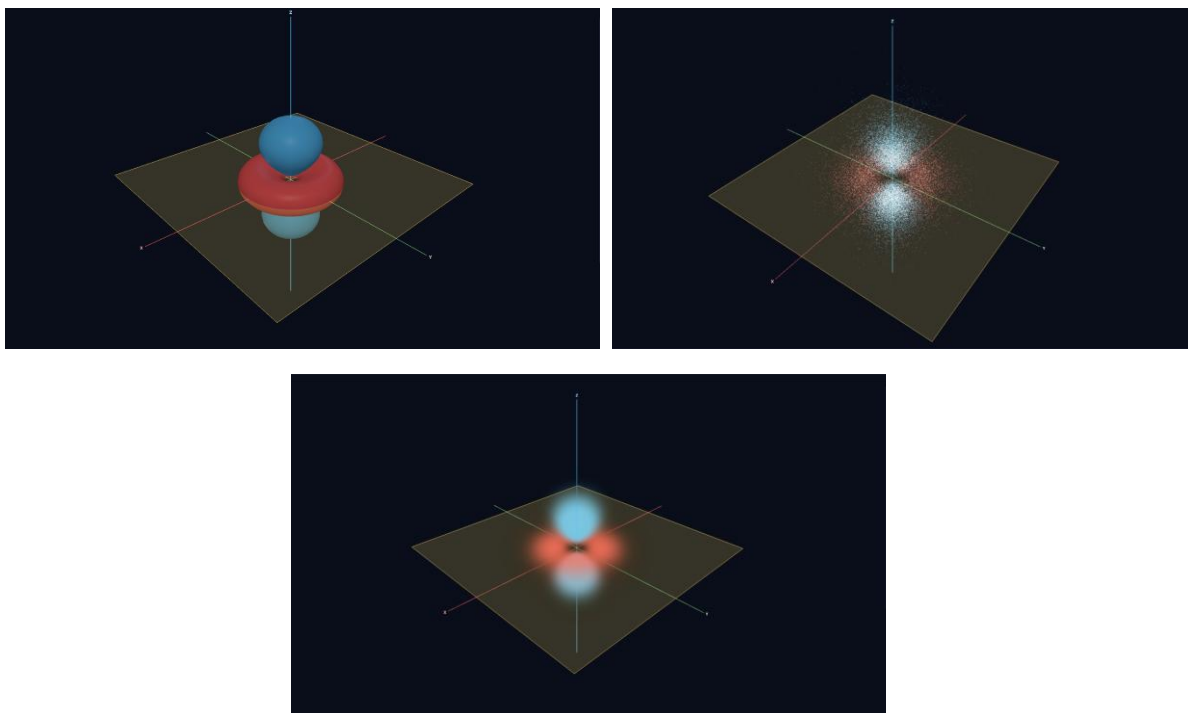


Fig.2 A row of three images of the orbital  $3d_{z^2}$  in volume, isosurface, and point-cloud modes side by side, to show they are three views of one thing.

## 2.4 The two supporting plots

Beneath the 3D view, two plots show information that is hard to read off the 3D shape alone.

- The radial distribution plot answers the question how far from the nucleus is the electron likely to be? Its peak marks the most probable distance. For the  $1s$  orbital this peak sits at almost exactly one Bohr radius, the very distance Bohr proposed in 1913 for the electron's orbit. The tool quietly recovers a century-old result from the modern probability picture, which makes a memorable teaching moment.

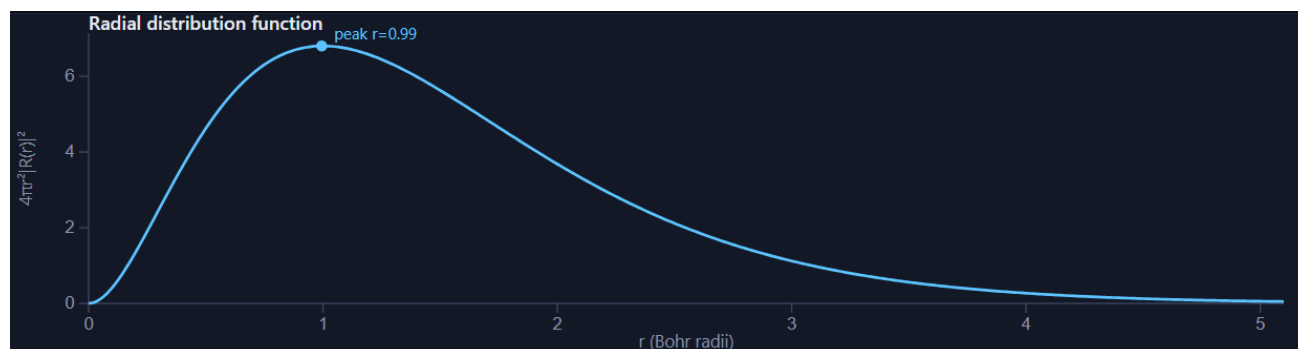


Fig.3 The radial distribution plot for  $1s$  with the peak at  $r \approx 0.99$  annotated.

- The slice plot cuts the orbital along a plane and shows the wavefunction on that slice as coloured contours. This makes nodes surfaces where the probability is exactly zero visible; as the lines where blue meets red or where colour fades to nothing. The classic demonstration is slicing the 2p orbital across its waist: the slice goes blank, because the entire plane is a node.

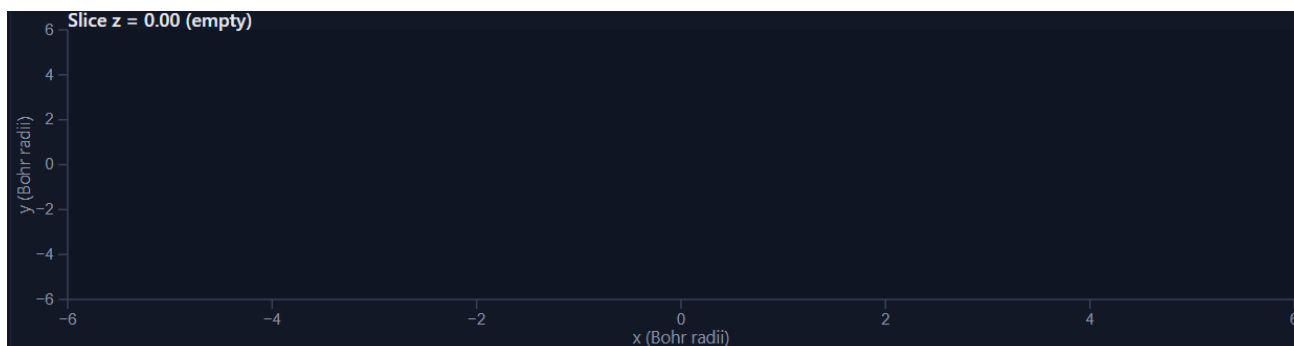


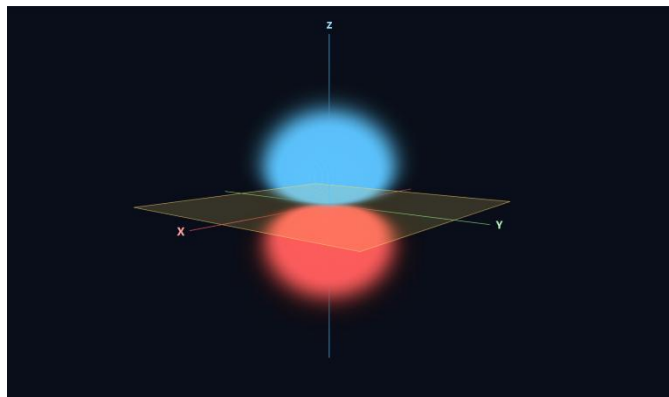
Fig.4 the 2p slice at  $z = 0$  going empty.

## 2.5 A short guided tour

To give a sense of how a beginner would use the tool, here is a short walkthrough through three orbitals. Each one teaches a different idea, and together they build the mental shift from "an orbital is a fixed shape" to "an orbital is a probability cloud with structure."

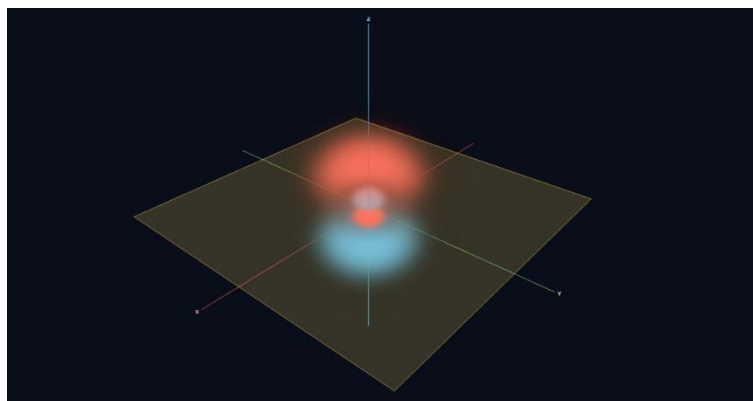
Stop one: the 1s orbital. The student opens the tool and switches to the 1s orbital. What they see is a single soft blue glow, brightest at the centre and fading smoothly outwards in every direction. This is already a useful first step: there are no edges, no clean boundary, no orbit just a cloud that gradually gets sparser the further out you look. Beneath the 3D view, the radial distribution plot shows a single hump with its peak marked at  $r \approx 0.99$  Bohr radii. That number is worth pausing on. It is, to two decimal places, the radius of the orbit that Bohr proposed in 1913 for the electron in hydrogen. Bohr's planetary picture is wrong, but the most probable distance from the nucleus in the modern probability picture turns out to be almost exactly the radius he postulated. The tool quietly recovers a century-old result from a completely different theory, and that coincidence is a memorable hook for a student.

Stop two: the 2p orbital. Switching to 2p<sub>z</sub> immediately introduces two new ideas. First, the orbital is no longer round: it has two distinct lobes, one above the nucleus and one below. Second, and more importantly, the two lobes are coloured differently blue on top, red on the bottom. The colour is not decorative. It encodes the phase of the wavefunction: the top lobe has positive phase; the bottom has negative phase. Most textbook pictures hide this, but it is the property that decides whether atoms bond constructively or destructively when their orbitals overlap, and it is best introduced early. The 2p orbital also lets us demonstrate a node a place where the electron will never be found. Switching the slice tool to the XY plane and setting the position to zero, the 2D slice plot goes blank. The entire xy-plane is a node:  $\psi$  is exactly zero everywhere on it. Move the slice slightly above zero and the slice fills with blue; move it slightly below and it fills with red. The phase flip across the node is now undeniable.



*Fig.5 The  $2p_z$  orbital rendered as a volumetric probability cloud. The blue and red lobes have opposite phase; the gap between them is the angular node where the electron has zero probability of being found.*

Stop three: the  $3p$  orbital, in volume mode. This is where the tool earns its keep. Switching to  $3p_z$  and selecting Volume mode reveals something almost no textbook ever shows inside each of the two big outer lobes, there is a small lobe of the opposite phase. The blue outer lobe contains a small red core, and the red outer lobe contains a small blue core. These inner lobes are the consequence of the radial node the  $3p$  wavefunction crosses through zero at  $r = 6$  Bohr radii, and the inner region has the opposite sign to the outer.



*Fig.6 The  $3p_z$  orbital in volume mode. The small inner lobes have the opposite phase to the outer ones, a direct visualization of the radial node at  $r = 6$  Bohr radii.*

The radial distribution plot beside the 3D view tells the same story from a different angle: a smaller hump near the origin, a sharp red dashed line at  $r = 6$  marking the node, and the main peak at  $r \approx 12$ . Three views the 3D cloud, the radial plot, and what the student already understands from the simpler orbitals line up to take a single coherent picture of why  $p$  orbitals get more complicated as  $n$  increases.

After this three-orbital tour, a beginning student has met every essential idea the tool is built to teach: orbitals are probability clouds rather than orbits, they have phase as well as shape, and their structure follows orderly rules with  $n$  and  $l$ . The remaining orbitals in the dropdown are explorable on the student's own terms.



### 3. Visualisation design

This section explains the design decisions behind the visualisation the choices made so that the tool teaches well. These choices are where the visualisation thinking lives, as distinct from the graphics implementation in §4.

#### 3.1 Why three render modes

No single representation teaches everything. Each mode corrects a different misconception and reveals a different aspect of the orbital. Offering all three, switchable in place, lets a student triangulate a single mental model from three views. The table below compares them.

| Render mode        | How it works                                                              | Strength (teaching)                                                                          | Weakness                                                  |
|--------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <b>Isosurface</b>  | A solid mesh extracted at a chosen probability contour (marching cubes)   | Clean, recognisable textbook shape; easy to read at a glance                                 | Shows only one contour; hides how density varies smoothly |
| <b>Volume</b>      | A glowing cloud produced by ray-marching through the density field        | Most honest picture of a probability cloud; shows full structure including faint inner lobes | Softer, harder to judge exact extent; more GPU-intensive  |
| <b>Point cloud</b> | Dots scattered with density proportional to the probability (Monte Carlo) | Communicates the probabilistic interpretation viscerally                                     | Noisy; needs many points; no hard surface to read         |

In practice, each mode earns its place at a different moment in a student's learning. **Volume rendering is the first stop:** it is the mode to use when a student is still being convinced that an orbital is not a solid object. The soft, edgeless glow of a volume render is hard to interpret as a shell, and that visual ambiguity is exactly the point. **Isosurface is the second stop:** once the student has accepted that orbitals are clouds, the isosurface offers a single clean shape they can memorise and recognise across textbooks. The threshold slider also teaches a subtle lesson moving it changes the shape, which means the textbook shape itself is a choice, not a real edge. **Point cloud is the third stop, and a special-purpose tool:** it is the mode to reach for when the student needs to *believe* the probabilistic interpretation, or when they need to see structure that an opaque outer surface would hide. The 2s orbital is the canonical example: its inner radial node is invisible through a solid isosurface, but the point cloud's individually transparent points let the inner shell of probability show through.

#### 3.2 Why encode phase in colour

The decision to colour positive and negative regions of the wavefunction in different colours is one of the most consequential design choices in the tool. Most introductory textbooks do not show phase at all they render orbitals as monochrome shapes, and the sign of the wavefunction is left implicit or never mentioned. The result is that students often complete an entire semester of inorganic chemistry without realising that the lobes of a p orbital have opposite signs, even though that fact is what makes molecular bonding work. Showing phase early, before it is needed, plants a seed that the student will draw on when they meet bonding and antibonding orbitals later.

The specific colours were chosen with two constraints in mind. The first is perceptual: blue and red sit at opposite ends of the cool/warm axis, which the human visual system distinguishes very robustly, so the two phases read as categorically different at any size or distance. The second is accessibility: the most common form of colour-blindness affects red and green together, so a red-and-blue pair remains distinguishable for almost all viewers. The same two colours are used consistently across every view in the tool the 3D mesh, the



volume cloud, the point cloud, and the 2D slice contour, so that the student learns one colour rule once and applies it everywhere.

### 3.3 Why three coordinated views

The tool deliberately presents three views of the same orbital at the same time: the 3D rendering at the top, the radial distribution plot at the bottom-left, and the slice contour at the bottom-right. This is an application of the coordinated multiple views principle from information visualisation: when a single object has multiple meaningful properties, showing each property in the view best suited to it and keeping all the views in sync lets the viewer move between perspectives without losing the thread.

In this tool, each view answers a question the other two cannot. The 3D view answers what does it look like? It conveys overall shape, orientation, and the spatial relationship between lobes. The radial distribution plot answers how far from the nucleus? It makes radial nodes visible as zeros and most-probable radii visible as peaks quantitative facts that are hard to read off a 3D cloud. The slice plot answers what is happening on this plane? It exposes angular nodes and the boundary between positive and negative phase regions, both of which are invisible from outside the 3D surface. Switching the selected orbital updates all three views at once, so the student sees how the same change reshapes every view simultaneously. This coupling is the part of the tool that turns it from a viewer into a learning instrument.

### 3.4 Designing for the beginner

Several smaller decisions throughout the tool are driven by the audience. The orbital dropdown lists orbitals in the order students meet them in class, 1s, 2s, then the 2p family, then 3s, 3p, 3d, so that exploring the dropdown mirrors the progression of a chemistry lecture rather than imposing a new ordering. Each rendering uses the chemistry convention of Z pointing up rather than the graphics convention of Y up, so that "the p<sub>z</sub> dumbbell is vertical" matches what the student has seen in their textbook. The axes are always labelled X, Y, and Z in their standard colours, so the student can orient themselves regardless of how they have rotated the view. Sensible defaults a 2p<sub>z</sub> orbital on opening, the isosurface threshold at 18%, the slice plane at the origin gives a student something interpretable to look at before they touch any controls, removing the empty-screen problem that defeats many interactive tools. Finally, the threshold slider is itself a teaching device: dragging it shrinks or inflates the visible shape, which forces the student to confront the fact that the "boundary" of an orbital is a choice of probability cut-off, not a physical surface.

## 4. Computer-graphics techniques used

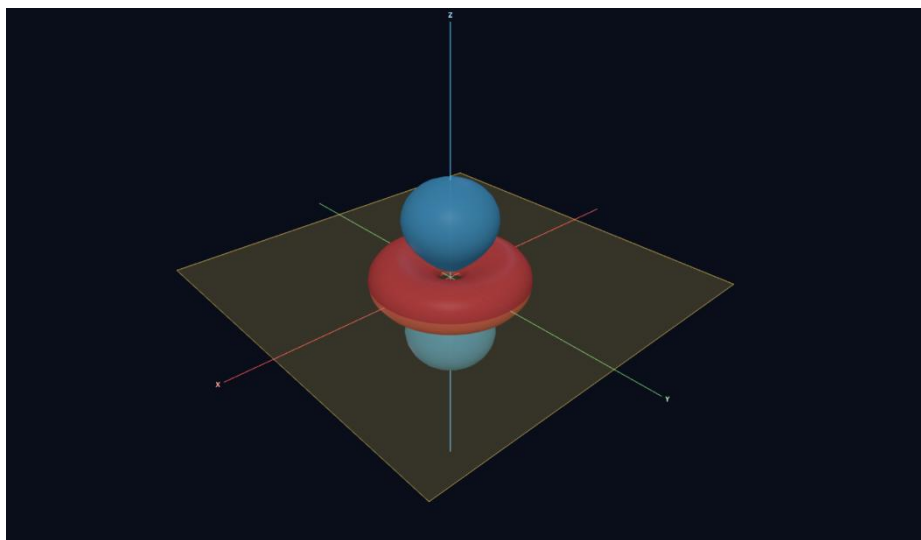
This section names the graphics techniques that make the tool work and explains, briefly, how each serves the visualisation. Per the project's framing, the emphasis is on the role each technique plays rather than on derivations or code; only the minimum necessary detail is given.

### 4.1 Turning a field into a surface: marching cubes

The wavefunction is a continuous field of values in 3D space where every point has a number associated with it, and the orbital we want to draw is the set of points where that number reaches a chosen level. The classical way to extract such a surface from a continuous field is the marching cubes algorithm, first published by Lorensen and Cline in 1987 and now the standard surface-extraction method in scientific visualisation and medical imaging.

The intuition is simple. The space around the nucleus is divided into a regular grid of small cubes. For each cube, the eight corner values of the field are inspected and classified as either above or below the chosen threshold. The pattern of "above" and "below" corners, there are 256 possible patterns, though only 15 are distinct after accounting for symmetry determines how the surface threads through that cube. A lookup table gives the triangles that make up the local surface, and a short interpolation places each triangle vertex at the exact point on the cube edge where the field value equals the threshold. Marching from one cube to the next and stitching the triangles together produces a watertight triangle mesh that approximates the surface across the whole volume.

The tool implements marching cubes from scratch in JavaScript rather than relying on a library, and extends the classical algorithm to handle the signed wavefunction by running it twice per cube once for the positive-



*Fig.7 The 3d\_z<sup>2</sup> orbital rendered as a marching-cubes isosurface. Both phases are extracted in the same pass; the positive lobes (blue) and the negative torus (red) are tagged by sign for separate coloring.*

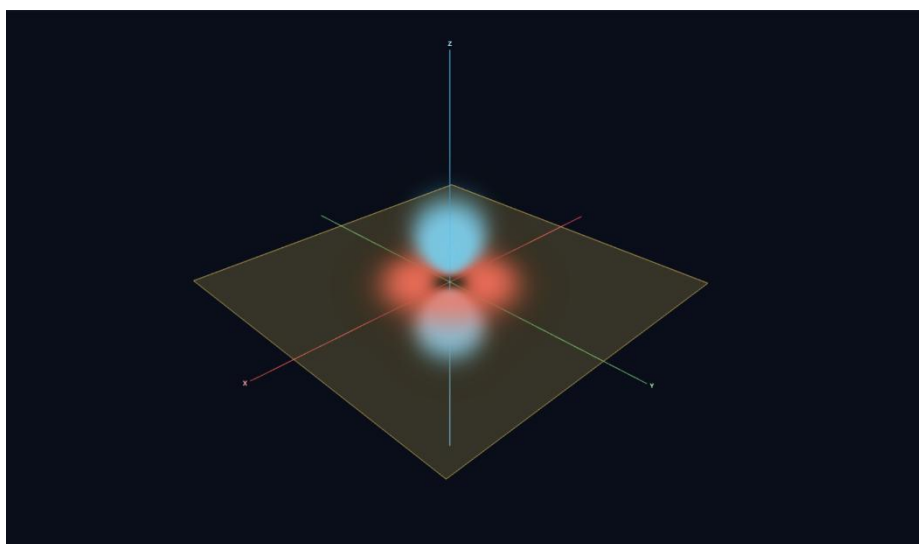
phase contour and once for the negative-phase contour, and tagging each triangle with its sign so the renderer can colour positive and negative lobes differently. Surface normals are computed not from the triangles themselves but from the gradient of the wavefunction at each vertex, which gives smoother shading than averaged face normals would. For a typical orbital on a  $48^3$  grid, marching cubes produces around 5,000 triangles in 50 to 150 milliseconds, which is fast enough to feel instant when the user switches orbitals.

## 4.2 The glowing cloud: volume ray-marching

Where the isosurface renders a single chosen contour of the wavefunction, the volume mode renders the entire density field at once. This is the most advanced rendering technique in the tool, and it is the closest to the family of techniques described loosely as "ray tracing": rather than rendering surfaces and computing where they reflect light, it sends a ray from the camera into the scene for every pixel and integrates the density along that ray.

The algorithm is called ray marching. The wavefunction is precomputed on a  $64^3$  grid and uploaded to the GPU as a three-dimensional texture. A fragment shader runs once per pixel on the screen, where it constructs a ray from the camera through that pixel, finds where the ray enters and exits the bounding cube of the orbital, and then steps along the ray in around 100 small steps. At each step it samples the 3D texture to read the local density, picks a colour based on the phase, and accumulates colour and opacity using front-to-back alpha compositing dense regions contribute strongly, sparse regions contribute weakly, and the final pixel colour is the integrated result along that line of sight. The shader applies an early termination when the accumulated opacity saturates, skipping work in the rest of the ray and keeping the renderer real-time even with hundreds of thousands of ray samples per frame.

The visual result is the most honest depiction of a probability cloud the tool can produce. The orbital has no hard edge anywhere: it brightens where the electron is likely to be found and fades smoothly into the dark



*Fig.8 The 3d\_z<sup>2</sup> orbital in volume mode. The cloud has no hard boundary density fades smoothly with distance from the nucleus.*

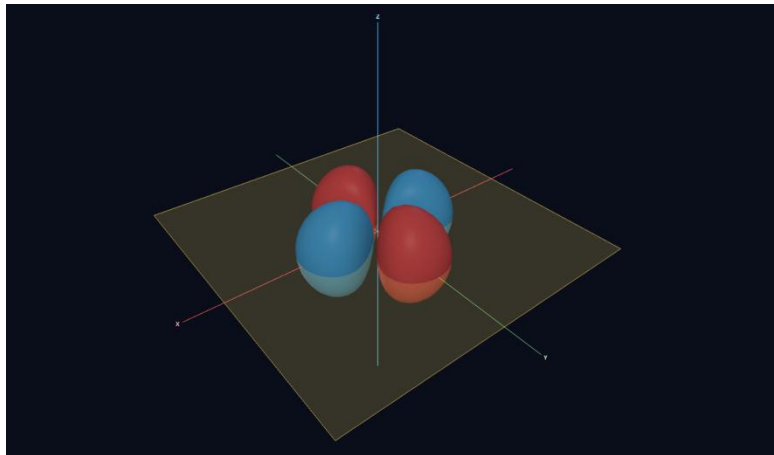
background. The phase colouring is preserved in 3D positive regions glow blue, negative regions glow red and structure that an isosurface hides is suddenly visible. The 3p<sub>z</sub> orbital is the strongest example: its inner counter-phase lobes, which sit inside the outer ones and would be occluded by a solid mesh, are clearly visible in the volume render.

## 4.3 Making it look three-dimensional: lighting and shading

**The isosurface is a triangle mesh, and without lighting it would render as a flat silhouette in a single colour.** To make it read as a curved 3D object, the tool applies the Phong reflection model, the standard local lighting model from the lighting and shading lectures which combines an ambient term (uniform background light), a diffuse term that brightens surfaces facing the light, and a specular term that places glossy highlights where the surface mirrors the light toward the camera. This combination gives the curved lobes of an orbital the soft

sheen of a piece of polished glass, which communicates curvature far more effectively than a uniform colour fill.

The scene uses a classical three-point lighting arrangement borrowed from photography and film. A bright white key light from the upper right defines the orbital's main form; a softer, cool-tinted fill light from the lower left lifts the shadowed side so it does not go completely dark; and a warmer rim light from behind picks out the edges of the lobes and separates them visually from the dark background. Each light's colour and intensity is chosen deliberately, not by default. A second choice that affects perceived smoothness is the type of shading interpolation. With flat shading, every triangle has a single normal and the mesh looks faceted, like



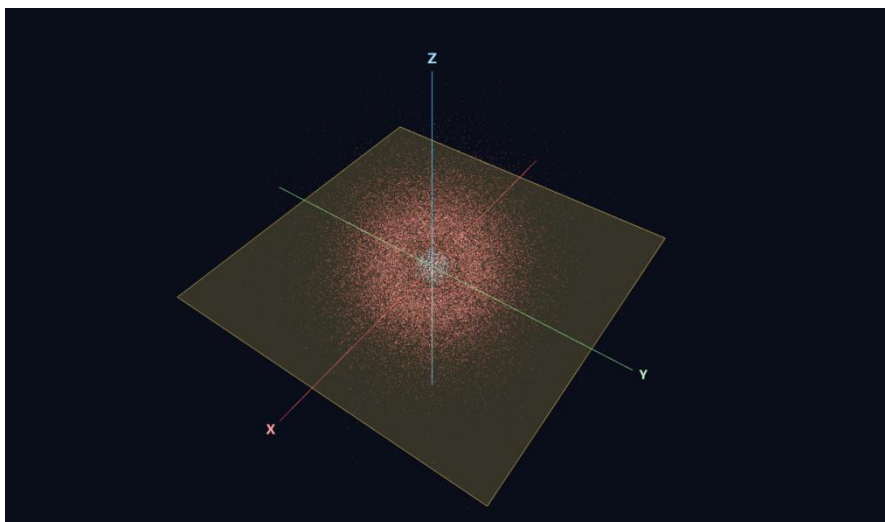
*Fig.9 The  $3d\_x^2-y^2$  orbital lit with three-point lighting and Phong shading. The specular highlights on each lobe communicate curvature; smooth shading hides the marching-cubes facets.*

a low-poly model. The tool uses smooth shading, in which a normal is stored at every vertex and interpolated across the triangle's interior, so the underlying marching-cubes facets disappear and the surface looks continuous. The vertex normals themselves are not computed from triangle geometry but from the gradient of the wavefunction at each vertex, which gives a normal that is correct with respect to the underlying field rather than to the discretisation grid.

#### 4.4 Colour, transparency, and the point cloud

The phase colouring described in §3.2 is implemented at the shader level. Each vertex of the isosurface mesh carries a colour attribute (blue or red) chosen by the sign tag from marching cubes; the volume shader picks a colour per ray-march step based on the sign read from the 3D texture; and the point cloud assigns a colour per point based on the sign of the wavefunction at that point. Three different rendering paths, one consistent colour rule.

The point cloud uses additive blending with depth-writing disabled. Additive blending sums the contributions of overlapping points rather than choosing one over the other, which is the correct behaviour for a probabilistic "glow": where many points fall close together, the brightness builds up naturally. Disabling depth-writing prevents the transparent points from occluding one another in arbitrary order, which would otherwise cause unsightly artefacts. The points themselves are placed by rejection sampling random candidate points are proposed uniformly in the bounding box and accepted with probability proportional to  $|\psi|^2$ , so the points that remain are distributed according to the probability density.



*Fig.10 The 2s orbital rendered as a point cloud. The inner shell of red points, corresponding to the radial node where the wavefunction flips sign is visible through the transparent outer blue cloud. The same structure is hidden behind an opaque isosurface at any threshold.*

The isosurface uses standard alpha blending at 92% opacity, with double-sided rendering. Double-sided rendering means both the front and the back faces of each triangle are drawn, which matters when the camera can be inside the surface as it can for s orbitals, where the user might want to zoom inside the outer shell to see what is there. The slice-plane indicator inside the 3D view uses very low opacity (around 18%) so it remains a faint guide rather than obscuring the orbital it is supposed to be cutting through.

#### 4.5 Coordinates, camera, and axis labels

The scene uses a standard right-handed coordinate system with the Z axis pointing up, the chemistry convention rather than the Y-up convention typical of computer-graphics libraries. This is enforced explicitly at the camera level, which costs nothing and ensures that "the p<sub>z</sub> orbital points up" matches what a student reads in their textbook. A perspective camera with a 45° field of view provides depth cues for the free-rotating orbitals, and an interactive controller binds mouse drag to orbiting the camera around the orbital, mouse wheel to zooming, and right-drag to panning. The camera distance is reset to fit each orbital's size whenever the user switches selection, so the orbital is always framed without the user having to zoom manually.

The reference axes are drawn as three coloured line segments through the origin, red for X, green for Y, blue for Z, matching the universal RGB-XYZ convention used in essentially every 3D tool with text labels floating at the tips. Because WebGL has no built-in text primitive, each label is rendered by drawing the letter onto a small 2D HTML canvas, uploading that canvas to the GPU as a texture, and mapping the texture onto a sprite: a small flat plane that is rotated each frame to always face the camera. The result is a label that is always

legible no matter how the scene is rotated. This is a direct application of the textures-and-billboards material from the texturing lecture.

#### 4.6 The 2D plots

The two 2D plots beneath the 3D view, the radial distribution function on the left, the slice contour on the right, are drawn using the D3.js library and are kept deliberately lightweight compared to the 3D rendering. The radial plot samples the radial part of the wavefunction at a few hundred evenly spaced points, draws a line through them, and annotates the peak (the most probable radius) and any zero-crossings as radial nodes. The slice plot evaluates the wavefunction on a regular 2D grid in the chosen slicing plane and uses D3's contour generator to draw filled level sets of the result.

One small technique in the slice plot is worth naming. The wavefunction is signed, but D3's contour generator works on a non-negative scalar field. The solution is to split the slice into two fields, one containing only the positive values (with zero everywhere else) and one containing only the absolute values of the negative regions and run the contour generator on each separately. The blue contours come from the first field; the red contours come from the second. The two sets of contours never overlap because the underlying values do not overlap, and the boundary between them traces out the orbital's nodes on the slice.

## 5. Discussion

### 5.1 Comparing the three rendering methods

The isosurface mode is the most decisive: it produces a single hard surface at a chosen threshold, which is the closest analogue to the canonical textbook picture. Its strength is also its weakness, it shows one contour and discards the rest, so the smooth variation in density is lost and structure hidden behind the contour is occluded. The threshold slider mitigates this by letting the user move the contour, but only one contour is visible at a time.

The volume mode is the least decisive and, paradoxically, the most truthful. It integrates the entire density field along each ray of sight, so nothing is hidden and the fall-off at the edges is rendered faithfully. The cost is that the boundary of the orbital is no longer crisp, so it is harder to extract a definite "shape" from. It is the right mode for breaking the misconception that orbitals are solid objects, and the right mode for revealing inner structure that an isosurface would occlude the 3p<sub>z</sub> inner counter-phase lobes are the cleanest example.

The point cloud mode is the most literal: each point is a possible location for the electron, and the density of points directly visualises the probability. It communicates the probabilistic interpretation more viscerally than either of the other two, and because each point is individually transparent it shows internal structure that the isosurface cannot. Its weakness is noise; the sampler must produce many tens of thousands of points before the orbital reads as a coherent cloud rather than a fuzzy blob.

In practice the three modes complement each other rather than compete, which is why the tool offers them as a toggle rather than asking the user to commit to one. A student exploring an unfamiliar orbital benefits from moving between all three views.

### 5.2 Performance

The tool is designed to feel instantaneous on a typical laptop GPU. Each rendering mode has its own cost profile.

The isosurface is the most expensive to prepare but the cheapest to render. Building the mesh requires sampling the wavefunction on a  $48^3$  grid (around 110,000 evaluations) and running marching cubes over the resulting voxels. For a typical orbital the result is around 5,000 triangles produced in roughly 50 to 150 milliseconds on a modern laptop, which is fast enough that the user perceives selecting a different orbital as essentially instantaneous. Once the mesh exists, the GPU renders it at full frame rate with no per-frame computation.

The volume mode reverses the cost profile. Building the volume is cheap, sampling the wavefunction onto a  $64^3$  grid takes around 10 milliseconds, and uploading the result as a 3D texture is essentially free but every rendered frame asks the GPU to march around 100 samples along a ray for every pixel on screen. This is feasible only because it runs entirely as a fragment shader on the GPU; doing the same work on the CPU would not be real-time. An early-ray-termination optimisation in the shader stops marching as soon as the accumulated opacity saturates, which substantially reduces work for pixels that hit the dense core of an orbital.

The point cloud uses rejection sampling, which is conceptually simple but has a poor acceptance rate at higher  $n$ . For the 1s orbital the sampler accepts a few percent of proposals; for 3d orbitals the acceptance rate drops below one percent. The total time still measures in a few hundred milliseconds because rejection-sampling each proposal is itself trivially fast, but it is the one place in the tool where the algorithm is visibly suboptimal.



A more sophisticated sampler using the radial distribution as an importance function, for example would accept dramatically more proposals per second. This is the obvious next optimisation if the tool needed to scale to larger orbital sets.

### 5.3 Limitations and what was left out

A few limitations are worth naming honestly. The orbital library covers  $n = 1$  through  $n = 3$  only; extending to  $n = 4$  and  $n = 5$  would require adding more hard-coded radial parts (a closed-form table extension, not a research problem) but was not done in the available time. The tool visualises hydrogen exclusively; multi-electron atoms require electronic structure methods well beyond the scope of an undergraduate graphics project and were never in scope. The volume renderer's density brightness needed an empirically chosen scale factor because the wavefunction's normalisation makes higher- $n$  orbitals naturally fainter per unit volume; a proper per-orbital auto-scaling would be a clear next refinement.

One feature was prototyped and removed. A side-by-side comparison mode that would render two orbitals at once across a split view was attempted using the WebGL scissor test, but encountered persistent issues with camera framing across the two viewports and interactions with the slice-plane indicator. After several debugging passes it was descope'd to protect time for the report and demo video. The functionality is described in this report and could be re-added in a future iteration; the engineering decision to cut it was driven by time, not by any fundamental obstacle.

### 5.4 What I learned building it

A few lessons surfaced during the build that are worth recording.

The first is that shader debugging is qualitatively harder than CPU debugging. A bug in JavaScript surfaces as a stack trace with a line number; a bug in GLSL typically surfaces as a black screen with no message at all. The volume shader took noticeably longer to get right than any CPU-side module, and the only useful debugging technique was inserting deliberately wrong "sentinel" colours to confirm the shader was running at all. This is a real working habit, not just a one-off observation.

The second is that small JavaScript mistakes can have large effects. Two memorable bugs during development were a CSS selector mistake (`#mode-row.mode-btn` versus `#mode-row .mode-btn`, where the missing space silently selects nothing) and a temporal-dead-zone error from declaring state variables after the event handlers that referenced them. Each bug was a single-character or single-line fix once located, but each cost real time to track down. The lesson is to lay state out in one block at the top of a module and to verify selectors interactively rather than by inspection.

The third, and most useful for this report's framing, is that choosing the right representation is itself a visualisation skill, distinct from implementing it. Realising the 2s radial node would be hidden by an opaque isosurface but obvious in the point cloud was a representation choice, not a graphics one — and it is what distinguishes a viewer from a teaching tool.

## 6. Self-assessment

Per the task sheet, this self-assessment is reflective: it discusses how the work meets the minimum requirements and what was done beyond them.

### 6.1 Meeting the minimum requirements

The task sheet specifies that a graphics-project report should consist of the computer-generated imagery, a discussion, and a self-assessment, with code and animations submittable as separate files. This report contains all three required components. The interactive tool itself is the centre of the project: imagery from it is embedded throughout the document as evidence of the techniques discussed, and the full source code is submitted alongside this report as a separate archive (see Appendix A). The discussion sections cover the techniques used, the design choices behind them, and the trade-offs between alternative methods. This self-assessment is the third required component.

The project is submitted as a full project under §2 of the task sheet, demonstrating a broad range of advanced graphics techniques and applying them to a clear communicative purpose. It is not a mini project or a micro project; the scope, the number of techniques used, and the depth of engagement with each one would not fit those narrower categories.

### 6.2 Evidence of advanced skills and understanding

The rubric awards higher grades for evidence of advanced skills and understanding. The table below maps each lecture topic to where it appears in the tool. The discussion that follows explains how these techniques were not merely used but chosen deliberately, with awareness of the alternatives.

| Lecture topic                   | How it appears in the tool                                                                                                                                                                                                                           |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Coordinates and transforms      | Atomic-unit coordinate space, Z-up chemistry convention, labelled X/Y/Z axes                                                                                                                                                                         |
| Cameras, projection, visibility | Perspective camera with interactive orbit/zoom/pan; per-orbital framing                                                                                                                                                                              |
| Textures                        | Axis labels rendered to a canvas and mapped onto camera-facing sprites; 3D texture storing the density field for the volume renderer                                                                                                                 |
| Software rasterisation          | Marching cubes converts the continuous field into a triangle mesh that the GPU then rasterises                                                                                                                                                       |
| Light and shadow / 3D effects   | Three-point lighting; additive blending for the point cloud; double-sided surface rendering                                                                                                                                                          |
| Lighting and shading            | Phong reflection model on the isosurface; smooth per-vertex normals from the field gradient                                                                                                                                                          |
| Shadows and occlusion           | Depth testing and visibility resolution throughout; double-sided surface rendering and disabled depth-writing to composite translucent geometry (point cloud, slice plane) in the correct order. Cast shadows deliberately omitted (see note below). |
| Colour and vision               | Cool/warm phase colouring (blue positive, red negative); contrast against a dark background; conventional axis colours                                                                                                                               |

One row in the mapping deserves an honest qualification. The tool resolves occlusion fully; depth testing, double-sided surfaces, and explicit depth-write control so that transparent geometry composites in the right order, but it does not cast shadows. This was a deliberate design decision rather than an omission. A shadow would imply the orbital is an opaque solid casting light onto a surface, which directly contradicts the central teaching point of the tool: that an orbital is a diffuse probability cloud with no hard edge. Introducing shadows would have undermined the visualisation it was meant to support.

Beyond the breadth shown in the mapping table, several specific pieces of work go beyond what would constitute a minimum-effort graphics project.

**Marching cubes was implemented from scratch.** The 256-entry edge table and triangle table were entered by hand, the algorithm was extended to handle signed scalar fields with phase tagging (running the algorithm twice per cube, once for each sign), and the surface normals were computed from the gradient of the wavefunction rather than from triangle geometry. None of this was lifted from a library.

**The volume renderer is a custom GLSL fragment shader.** It implements ray-box intersection by the slab method, front-to-back alpha compositing, and early-ray termination as a performance optimisation. The technique is the closest in the tool to ray tracing it is ray casting through a participating medium and engages directly with one of the alternative project topics suggested in §3.3 of the task sheet.

**Three rendering modes share one wavefunction.** A single underlying scalar field is represented in three completely different ways (a triangle mesh, a volumetric texture sampled along rays, and a Monte-Carlo point set), each with its own appropriate transparency, blending, and lighting choices. This is the "comparison of methods" pattern that the task sheet identifies as a hallmark of a strong graphics project.

**Three coordinated views remain consistent.** The 3D view, the radial distribution plot, and the slice contour are kept in sync through a single orbital state, and the phase colour scheme is applied uniformly across all three. This is the coordinated-multiple-views principle from information visualisation applied carefully and consistently.

**Numerical correctness has been verified against textbook values.** The radial distribution peaks reproduce known most-probable radii to two decimal places (0.99 Bohr radii for 1s where the textbook value is 1; 4.01 for 2p where the textbook value is 4; 9.03 for 3d where the textbook value is 9), and the radial node of 3p appears at exactly  $r = 6$  as expected. The tool is not just visually plausible but quantitatively right.

### 6.3 What I would do differently

Two improvements would have made the project run more smoothly. The first is starting the report in parallel with the code rather than after it. Writing about a technique while it is fresh ideally on the same day it is implemented produces clearer prose than reconstructing the reasoning a week later. The second is budgeting more time for the volume renderer specifically, on the grounds that shader work is harder to debug than CPU code and therefore needs a wider time margin than its line count suggests.

## 7. Conclusion

This project set out to give a beginning student of quantum chemistry a way of seeing hydrogen orbitals not as flat textbook diagrams but as interactive three-dimensional probability clouds with structure, phase, and characteristic shape. The result is a browser-based tool that presents each orbital simultaneously as a glowing volumetric cloud, a clean solid surface, and a scatter of probability-weighted points, with two coordinated 2D plots showing the radial distribution and a cross-sectional slice. The visualisation is the goal; the graphics techniques marching cubes, volume ray-marching, Phong shading with three-point lighting, additive blending, phase-aware colour, textured-sprite labels are the tools that make the visualisation possible. The tool quietly recovers Bohr's 1913 orbit radius from the modern Schrödinger probability cloud as a side effect of being numerically correct, and it makes structural features such as radial nodes and phase flips obvious in ways that no static textbook figure can. The project demonstrates that computer-graphics techniques, applied with care to a well-chosen scientific subject, can produce educational tools that are both faithful to the underlying science and genuinely illuminating to the learner.

## References

- Lorensen, W. E. and Cline, H. E. (1987). Marching Cubes: A High-Resolution 3D Surface Construction Algorithm. *Computer Graphics (SIGGRAPH 87)*, 21(4), 163–169.
- Phong, B. T. (1975). Illumination for computer generated pictures. *Communications of the ACM*, 18(6), 311–317.
- Bostock, M., Ogievetsky, V., and Heer, J. (2011). D3: Data-Driven Documents. *IEEE TVCG*, 17(12), 2301–2309.
- Three.js documentation. <https://threejs.org/docs/> (accessed 2026).
- Griffiths, D. J. (2018). *Introduction to Quantum Mechanics*, 3rd ed. used only as the source of the hydrogen wavefunction shapes.

In addition to the above:

- COSC3000 lecture notes (Semester 1, 2026): coordinates and transforms; cameras, projection, and visibility; textures; software rasterisation; light and shadow; lighting and shading; shadows and occlusion; colour and vision.
- Three.js examples and reference shaders (<https://threejs.org/examples/>), consulted for the conventional patterns for Data3DTexture setup and ShaderMaterial usage.
- Claude was used for content filtration and refining the report. Also used for understanding of the concepts and helping in planning for the code.

## Appendix A: Code and How to Run it

### Live demonstration and source code.

A hosted version of the tool is available at <https://cosc-3000-orbitals.vercel.app/> for convenience. The full source code is publicly hosted on GitHub at <https://github.com/mostameues187/cosc-3000-orbitals> and is also submitted as a separate archive alongside this report. The hosted version is built from the same code, via `npm run build`, and served as static files through Vercel. All features described in this report, render-mode switching, the threshold and slice controls, and the coupled 2D plots are functional on the hosted version.

A short demonstration video is included with the submission as `cosc3000_demo_video.mp4`, and is also viewable online [here](#). The video provides a guided walkthrough of the interactive features described in Section 2. The project consists of the following files, organised in the `src/` directory:

`index.html`: The page skeleton, the canvas element, and the control panel markup.

src/main.js: Application entry point. Sets up the Three.js renderer, scene, camera, and lighting; owns the global state for the selected orbital, render mode, isosurface threshold, and slice configuration; wires the UI controls to the rebuild functions.

src/orbital.js: Closed-form hard-coded hydrogen wavefunctions  $\psi(n, l, m, x, y, z)$  for  $n = 1$  through 3, plus helper functions for the radial part and a list of available orbitals.

src/marchingCubes.js: The marching-cubes implementation, including the 256-entry edge and triangle lookup tables, the per-cube classification logic, vertex interpolation along edges, and the extension for signed (phase-tagged) scalar fields.

src/volumeRender.js: The GPU volume renderer. Includes the CPU-side grid sampling, the upload as a 3D texture, and the vertex and fragment shaders that perform ray-box intersection, ray marching, and front-to-back alpha compositing.

src/axes.js: Labelled X/Y/Z axes drawn as coloured lines with sprite-based text labels.

src/radialPlot.js: D3-based plot of the radial distribution function  $4\pi r^2 |R(r)|^2$  with annotated peaks and radial nodes.

src/slicePlot.js: D3-based 2D contour plot of the wavefunction on a chosen plane, with the signed field split into positive and negative parts before contouring.

src/slicePlane.js: The semi-transparent indicator plane drawn in the 3D view to show where the slice is being taken.

src/style.css: Layout and visual styling for the control panel, plot panels, and toggle buttons.

The development server prints a local URL (typically <http://localhost:5173/>). Opening that URL in any modern browser loads the tool. A production build can be produced with `npm run build`; the resulting static files in `dist/` can be served from any web server with no further runtime requirements.

## Appendix B: List of figures

- Fig. 1: The 2p orbital in volume mode, with labelled axes (§2.3) ... p. 2
- Fig. 2: 3d\_ $z^2$  shown in volume, isosurface, and point-cloud modes side by side (§2.4) ... p. 3
- Fig. 3: Radial distribution plot for 1s, peak annotated at  $r \approx 0.99$  (§2.5) ... p. 3
- Fig. 4: The 2p slice at  $z = 0$  going empty, demonstrating the angular node (§2.5) ... p. 4
- Fig. 5: The 2p\_ $z$  orbital as a volumetric probability cloud; opposite-phase lobes and the angular node between them (§2.5) ... p. 5
- Fig. 6: The 3p\_ $z$  orbital in volume mode; inner counter-phase lobes from the radial node at  $r = 6$  (§2.5) ... p. 5
- Fig. 7: The 3d\_ $z^2$  orbital as a marching-cubes isosurface; both phases extracted in one pass and tagged by sign (§4.2) ... p. 8
- Fig. 8: The 3d\_ $z^2$  orbital in volume mode; density fading smoothly with no hard boundary (§4.3) ... p. 9
- Fig. 9: The 3d\_ $x^2 - y^2$  orbital under three-point lighting and Phong shading; specular highlights conveying curvature (§4.3) ... p. 10
- Fig. 10: The 2s orbital as a point cloud; the inner radial-node shell visible through the transparent outer cloud (§4.4) ... p. 11